

# Questionnaire

# **COMP3900 Project Questionnaire**

## **Skillset**

- Darshan:
- Eric:
- Brandon: backend development, modular programming
- Thomas: Databases, mathematical modelling
- Jeffrey: Mathematical optimisation, backend development
- Daniel: distributed systems, machine learning, backend development

## **Preferences**

COMP3900 Project List r. P# - Title [Project Type] (Background Screening)

- 
1. P132 - Non planar 3D printing [Open Source Library] (Great)
  2. P92 - 3D LLM Testing Sandbox [Tooling] (?)

- 
3. P14 - Molecular Programming [Tooling] (Great)
  4. P121 - Trace Foot Sole [Backend] (Great)

- 
5. P85 - Teaching AI To Kids [Game] (Good)
  6. P120 - Clinical Support Games [VR Game] (Great)

- 
7. P135 - Digital ID For Payments [Backend] (?)

Question: Concisely describe your group's relevant knowledge and skills that make you suitable for your Preference n. You may discuss for example, any relevant courses, internships, projects, and skills.

## **Disciplines**

1. Artificial Intelligence (Machine/Deep Learning, NLP)
2. Computer Science and Algorithms
3. Software Development

**Template:**

Our team are all double majors in mathematics and computer science. We cover all fields of mathematics among the six of us: pure mathematics, applied mathematics and statistics. We have extensive experience in algorithmic thinking, mathematical modelling and machine learning, and between us, we have honed our skills by placing as national finalists in the International Mathematical Modelling Competition twice, interning on global payment systems developing machine learning models to detect fraud and scams and furthering research into feature importance and model governance to detect degrading performance among biased models that are currently being used company-wide as an integration into the in-house model hosting platform.

We also have extensive experience in backend development, distributed systems and competitive programming. Between us, we have honed our skills by placing in the International Collegiate Programming Contest regionals as observers, interning on social media platforms to extend the capabilities of distributed systems handling 100,000 messages every second with new message types, introducing machine moderation, introducing moderation hinting, introducing automated root cause analysis to understand false positive results, and handling new compliance laws.

As for our interpersonal and soft skills, we have leadership and communication skills through external involvement and founding of student societies, high school outreach, volunteering in subcommittee and director positions and handling requirements from many projects working with multiple clients simultaneously across teams, disciplines and regions. Handling requirements from many projects simultaneously, working with multiple clients across teams, disciplines and regions.

Courses: Higher Linear Algebra, Higher Theory of Statistics, Extended Algorithm Design & Analysis (some of us are tutoring it), Programming Challenges, Software Design and Architecture (tutoring), Database Systems, Computer Networks & Applications.

Languages: C, C#, C++, Java, Python, Go, JavaScript, TypeScript, PostgreSQL, GdScript, Bash, Unix, Unity, LaTeX

**Preference 1:**

Our team are all double majors in mathematics and computer science. We cover all streams of mathematics among the six of us: pure mathematics, applied mathematics and statistics. As a result, we are very familiar with linear algebra and 3D geometry in multiple contexts due to our diverse mathematical range. We have extensive experience in algorithmic thinking, mathematical modelling and backend development, and between us, we have honed our skills by placing as national finalists in the International Mathematical Modelling Competition twice, interning on social media platforms to extend the capabilities of distributed systems and competing in the International Collegiate Programming Contest regionals as observers. All members of our group have experience with backend development and are comfortable with programming in C and Python. As a result of this, we are comfortable working with version control tools, testing, and developing multi-module codebases that can easily be extended or modified.

We are very familiar and proficient with computational geometry from a rigorous competitive programming setting. We also have experience with creating visualisation programs, having programmed a visualisation program for a research task on the Angel and Devil problem by the same team, which is relevant to visualising generated toolpaths without physical hardware. We also all have extensive experience with software design and architecture, crucial for writing good modularised codebases, with two of us tutoring the Software Design and Architecture course that we all have taken. Additionally, between us, we have experience interning on global payment systems developing machine learning models to detect fraud and scams, involving gigantic codebases.

As for our interpersonal and soft skills, we have proven leadership and communication skills through internal involvement with and founding of student societies, high school outreach, volunteering in subcommittee and director positions and handling requirements from project managers working with multiple diverse client demands.

Ultimately, our experience in computational geometry, backend development, version control, and mathematics provides us with the necessary tools to perform well in this project.

**Preference 2:**

Our team are all double majors in mathematics and computer science, spanning across the disciplines of pure, applied and statistics. We've chosen the double major because of an interest in researching, developing, and applying mathematical knowledge. The team all have a background in advanced geometry from the courses Higher Linear Algebra and Higher Several Variable Calculus, and have applied it computationally in Programming Challenges and Extended Algorithm Design and Analysis (as tutors and students). Between us, we have placed as national finalists in the International Mathematical Modelling Competition twice, participated in the International Collegiate Programming Contest South Pacific regional finals, set up Ollama for local use, and interned on global payment systems with billions in revenue, developing a Python library for ML/AI model governance optimised to handle terabytes of samples (pandas and polars dataframes) every day.

Our team is also proficient in software engineering through coursework in Software Engineering Fundamentals, Software Design and Architecture (as tutors and students), Database Systems, Computer Networks & Applications, and Software Tools & Construction. We have expertise in backend development, distributed systems and CI, and collectively have built quiz platform prototypes in TypeScript, home labs deploying services in Docker Compose and, as interns, continuously integrating automatically tested requirements into a globally distributed social media platform with over a billion daily active users, including features calling in-house LLMs like proactive moderation of new message types, reactive arbitration of appeals and ML/AI hinting to human moderators.

Through involvement in student societies as subcommittee members, directors and founders, university outreach to high schools, university tutors to fellow undergraduate computer science students and internships at large international tech companies working with various cross-functional, cross-regional, and cross-disciplinary teams, the team has also developed strong interpersonal and communication skills.

**Preference 3:**

Our team consists of confident programmers who are all well-versed in several languages, including but not limited to Python. The programming skills covered collectively by our group effectively accommodate for this project's requirements, having been primarily developed through the completion and tutoring of university-level computer science courses such as Software Design & Architecture, Extended Algorithm Design & Analysis, and Software Tools & Construction, as well as industry experience at social media platforms involving work on distributed systems. In addition, our familiarity in dealing with structured data, particularly JSON, aligns nicely with the demands of this project.

Furthermore, our team is highly proficient in Git and general software testing/debugging, stemming from experiences in high-stakes collaborative settings related to app development. Computational geometry and creating virtualisation programs are also part of our team's strengths, with backgrounds in competitive programming, like placements in the International Collegiate Programming Contest Regionals as observers, and development of a virtualisation program and graphical user interface for an algorithm design research task on the Angel and Devil problem in combinatorial game theory. These technical skills will allow for us as a team to comprehensively understand the details of the project and seamlessly integrate our individual strengths to complete the tasks at hand.

As for our interpersonal and soft skills, we have proven leadership and communication skills through internal involvement with and founding of student societies, high school outreach, volunteering in subcommittee and director positions, handling requirements from project managers working with multiple diverse client demands and both private and group teaching experiences.

Ultimately, our experience with computational geometry, backend development, software architecture and systems, and structured data provides us with the necessary tools to perform well in this project.

#### **Preference 4:**

Our team are all double majors in mathematics and computer science, spanning across all the disciplines of pure, applied and statistics. We bring extensive experience in software design, algorithm development and data analysis. Between us, we have refined our technical skills through advanced coursework including Extended Algorithm Design & Analysis, Software Design & Architecture, Database Systems and Machine Learning which is further backed by internship experience in developing scalable systems and assessing machine learning models. Our competitive programming experience is also relevant as we have placed in the International Collegiate Programming Contest regionals as observers.

Our technical skillset is ideal for this project's core demands including computer vision, feature detection and video preprocessing. We are all highly proficient in Python and are competent working with MatLab which allows us to build efficient and rigorous software architectures for areas such as marker trackers and contact area detections. We also have experience with creating visualisation programs, having programmed a visualisation program for a research task on the Angel and Devil problem by the same team.

As for our interpersonal and soft skills, we have leadership and communication skills through external involvement and founding of student societies, high school outreach, volunteering in subcommittee and director positions and handling

requirements from many projects, working with multiple clients simultaneously across teams, disciplines and regions.

#### **Preference 5:**

Our team are all double majors in mathematics and computer science, spanning across all the disciplines of pure, applied and statistics. We bring extensive experience in software design, algorithm development and data analysis. Between us, we have enhanced our technical expertise via advanced coursework including Extended Algorithm Design & Analysis, Software Design & Architecture, Database Systems and Machine Learning. Outside university coursework, we have internship experience in building scalable systems and assessing machine learning models. Furthermore, we have competitive programming experience where we have placed in the International Collegiate Programming Contest regionals as observers.

Our technical skillset and knowledge that we bring to this project is greatly suitable for the set of core features. We are proficient in the required languages, including Python, TypeScript, and JavaScript, while also implementing efficient data structures and algorithms. We also have experience in web application development and building visualisation programs, where we have programmed a 2D visualisation program for an algorithm research task on the Angel and Devil problem by the same team. Furthermore, we share a strong passion for teaching students ranging from primary school to university coursework and are interested in integrating an intuitive learning environment into the student learning process.

As for our interpersonal and soft skills, we have leadership and communication skills through external involvement and founding of student societies, high school outreach, volunteering in subcommittee and director positions and handling requirements from many projects working with multiple clients simultaneously across teams, disciplines and regions.

#### **Preference 6:**

Our team is also proficient in software engineering through coursework in Software Engineering Fundamentals, Software Design and Architecture (as tutors and students), Database Systems, Computer Networks & Applications, and Software Tools & Construction. We have expertise in backend development, game development and continuous integration, and collectively have built backend prototypes for quiz platforms, games to visualise research problems, and interned on a globally distributed social media platform with over a billion daily active users.

As double majors in mathematics, our team have experience working on complex geometric problems in 3D space from the courses Higher Linear Algebra and Higher Several Variable Calculus, and applied it computationally in Programming Challenges and Extended Algorithm Design and Analysis (as tutors and students). Between us, we have placed as national finalists in the International Mathematical

Modelling Competition twice, participated in the International Collegiate Programming Contest South Pacific regional finals, and interned on global payment systems with billions in revenue, optimising analysis of terabytes of data daily.

Through involvement in student societies as subcommittee members, directors and founders, university outreach to high schools, university tutors to fellow undergraduate computer science students and internships at large international tech companies working with various cross-functional, cross-regional, and cross-disciplinary teams, the team has also developed strong interpersonal and communication skills. We also have family and friends working in the healthcare scene, and are well experienced with the working woes of everyday medical professionals.

### **Preference 7:**

Our team has a strong proficiency in programming and software engineering through competitive programming, backend development, distributed systems and coursework in software architecture, databases, and computer networks. Within our group, we have experience working on production-level systems that have handled 100,000 messages per second with new message types, introducing machine moderation, introducing moderation hinting, introducing automated root cause analysis to understand false positive results, and handling new compliance laws. Furthermore, within our group, we have experience in developing fraud and scam detection machine learning models, which gives us some exposure to security in real-world systems.

All members of our group have experience with backend development in a multitude of programming languages. As a result of this, we are comfortable working with version control tools, testing, and developing multi-module codebases that can easily be extended or modified. In addition, we are all familiar with using REST APIs, as well as having extensive knowledge in software architecture principles.

As for our interpersonal and soft skills, we have leadership and communication skills through external involvement and founding of student societies, high school outreach, volunteering in subcommittee and director positions and handling requirements from many projects working with multiple clients simultaneously across teams, disciplines and regions. Handling requirements from many projects simultaneously, working with multiple clients across teams, disciplines and regions.

# Kickoff Email

# **Student Intro Email Template to Client**

## **Subject**

Introduction from 3900-F13A-APPLE - Multi-axis and Non-planar Slicer for Robotic 3D Printing

## **Email Body**

Dear Louis Lamont, Richard Blackwell and Max Melamed

We hope you are well.

We are 3900-F13A-APPLE, and we have been allocated to work on "Multi-axis and Non-planar Slicer for Robotic 3D Printing" for COMP3900/COMP9900 this term.

## **About our team**

Our team members are:

- Darshan - [scrum master / backend developer: project management / documentation / computational geometry]
- Eric - [product owner / backend developer: 3D geometry, mathematical modelling]
- Daniel - [backend developer / QA: backend, infra, distributed systems, machine learning, devops, QA, tooling]
- Jeffrey - [backend developer / technical lead: backend development, mathematical optimisation]
- Thomas - [backend developer / visualisation: databases / mathematical modelling]
- Brandon - [backend developer / infrastructure: backend development, modular programming]

## **Our current skillset**

As a team, we have experience in:

- C/C++/Python/Java
- Software Design and Architecture
- Computational Geometry
- Linear Algebra / 3D Geometry
- Mathematical Modelling / Optimisation
- Git / Versioning

We are excited to work with you and would love to start by confirming the best way to communicate and arranging our first meeting.

## **Communication preferences**

Could you please let us know:

1. Which communication channel(s) you prefer (e.g., email, Teams, Zoom, Slack, Discord)?
2. Your preferred time for a first kickoff meeting (preferably during this week if possible but early next week if not)?
3. Your preferred schedule for regular meetings (e.g., weekly/fortnightly, and preferred days/times)?

We are happy to adapt to a time and communication style that works best for you.

## **First Meeting**

For our first meeting we have prepared the following items for discussion and clarification:

- Project Requirements and Expectations
- Expected content to be included in the Project Proposal

Please share your earliest convenience for the first meeting since we have to catch up with the proposal deadline.

Thank you, and we look forward to working with you this term.

Kind regards,

3900-F13A-APPLE

COMP3900/COMP9900 Project Team

Eric Hong – z5588972@ad.unsw.edu.au

Ph: 0478 041 528

# Proposal



# (Client) Requirements

- Is this decision problem or optimisation (yes no we can do this vs gimme fastest or most efficient or least amount of lifts)

R: more of an initial MVP, current software doesn't exist yet really, there are enterprise softwares that are behind paywalls, slicing in Rhino (CAD tool) and Grasshopper. Looking for something that can initially supplement current workflows. Currently, found some initial libraries, some early results. Slicing tool that can generate more complex toolpaths compared to fixed Z slicing (planar slicing).

- What traits about the printing process are you looking to prioritise, or are you looking to generally balance these?
  - Surface quality
  - Total print time
  - Inter-layer cooling

R: we just want something out there, generally looking to balance traits. Prioritising a parameter is later on down the line.

- We will at least prototype in python. Will C be required?

R: Our choice of language.

- Can we use other open source libraries for development?

R: Our freedom, whatever we think we need.

Extra: getting both teams to work on slightly different things.

- How is visualisation occurring?
  - Separate web application
  - Separate desktop application
  - Within the library

R: web-based interface for visualisation.

- What exactly do they want for the visualisation?
  - Smth that can be paused / played / rewind

R: our choice

- any preferences for visualisation library/software? Or will anything do
- To the client, do you have a specific preference on what visualisation framework you'd like the code to be demoed on?
  - What to deploy on
- See if they want changes to our current API's inputs and outputs
  - Load: constructor, receives print model, robot and tool.
  - Compute: compute toolpath, receives print parameters and strategy
  - Visualise: run visualisation with existing toolpath if exist.
  - Export: return commands, receives output file type.

R: ABB models to be sent by client.

- If our project is successful, will our software be tested on actual hardware?

R: definitely, hopefully pretty soon

- Exactly what nozzle and robot we are using (we're modularising input but want to work with something for now)

Extra: We can assume the user has a decent understanding of the print parameters.

# Milestones

## **Sprint 1**

- Darshan: inverse kinematics (robot path)
- Eric: tool + print parameters (tool path)
- Daniel: pipelines and integration tests
- Thomas: visualisation
- Brandon: infra with all in out types
- Jeffrey: tooling, design

Design

## Rough design

### Rough flow:

Input (surface-defining file, STL, OBJ, or NURBS via CAD interchange, and robot and tool parameters) → file processing → toolpath generation (main software: segmentation, deposition, collision checking, etc) → toolpath exportation (to RAPID and KRL)

### Data Flow:

1. Load:
  - a. Receive inputs for print, robot and tool.
  - b. PrintInputStrategy: print -> PrintModel
  - c. RobotInputStrategy: robot -> RobotModel
  - d. ToolInputStrategy: tool -> ToolModel
2. Compute:
  - a. Receive inputs for PrintParameters and our PrintStrategy.
  - b. PrintStrategy determines the PrintPath (basically where is material getting printed.
  - c. CollisionScene containing CollisionModels will contain any existing CollisionModels, as well as initialising the print's CollisionModel to empty.
  - d. ToolPath (all positions where the tool can be) will be computed, tracing the PrintPath and taking into account PrintParameters and CollisionScene.
  - e. RobotPath will be computed, tracing the ToolPath and taking into account CollisionScene, selecting any possible solution from the ToolPath.
3. Visualise:
  - a. Open a separate program/window.
  - b. Visualisation takes in RobotPath + ToolPath + PrintPath to generate video.
  - c. Allows for pause, play, speedup, jump to (specific instruction), rotate, pan, zoom.
4. Export:
  - a. Export RobotPath based on ExportStrategies.

## Classes/Objects

## API Documentation

Infra

# Mesh vs Parametric Surfaces

**Mesh Format:** surface is broken into flat triangles (or polygons), approximates continuous surface normals and curvature

**Parametric Surface Representation:** surface defined by mathematical curves/patches

## STL (Stereolithography)

**Type:** mesh format

**What it is:** flat list of triangles, each with 3 vertices and a normal vector. Does not include colour, units, metadata, or topology (no explicit connection between triangles).

**Parsing:** binary STL follows 50-byte-per-triangle format. Libraries: `numpy-stl`, `trimesh`, `assimp` (Python/C++).

### Challenges for current use case

- **No inherent connectivity between triangles:** adjacency must be reconstructed in order for path segmentation/continuity (determine which triangles share edges).
- **Per-triangle orientation is discontinuous due to faceted normals:** every point within a given triangle (center, edges, corners) gets reported as having the *exact same normal*, since each triangle is *perfectly flat by definition* (three points define a plane, plane has exactly one normal vector). Being an approximation, the real surface might be smoothly curving across that entire region. **Why this matters:**
  - **Discontinuous Orientation Commands:** as the toolpath crosses from one triangle to the next, the optimal/ideal nozzle angle can jump instantly (even for a very small distance of travel). This forces the controller to drastically slow down at every boundary (decreases print speed) or produces jerky, non-smooth motion which reduces print quality and is bad for the hardware.
  - **Path-dependent Artifacts:** triangulation is somewhat arbitrary (the same curved surface can be meshed into multiple different valid configurations, it depends on the CAD export settings). This means that the exact normal at a given point in space (x,y,z) relies on the *way that given STL was triangulated*, and not on *the true geometric shape of the surface*. The same part of a surface can be exported at different mesh resolutions, which produces inconsistent orientation results.
  - **Coarse Tessellation Problem Amplified:** low-poly STL exports (large and sparse triangles) result in even more significant orientation jumps. High-res STL exports approximate smooth normal fields more closely, but leads to huge file sizes and more individual triangles to process for segmentation and collision checking.

The solution is to perform **vertex normal interpolation**, where for each vertex, you average the face normals of all triangles sharing that vertex (weighted by triangle area or angle at that vertex for more accuracy), and interpolate (using

**Phong-shading-style** interpolation, etc) to create a smoothly varying normal at any arbitrary point inside a given triangle.

- **No guarantee of watertightness:** practically, STLs usually have imperfections (gaps, non-manifold edges, duplicate vertices) so a repair/validation pass using appropriate library functions is needed before segmentation.

### Binary STL File Format

[80 bytes] Header (arbitrary, usually ignored)  
 [4 bytes] Number of triangles (unsigned 32-bit integer)  
 [triangle 1]  
 [triangle 2]  
 ...  
 [triangle N]

### Per-Triangle Layout

- Face Normal Vector ( $n_i, n_j, n_k$ ): 3 x 32-bit float
- Vertex 1/2/3 ( $x, y, z$ ): 3 x 32-bit float
- Attribute byte count: unsigned 16-bit integer

### Vertex Ordering and Normal (Right Hand Rule)

The three vertices of a triangle must be listed in **counter-clockwise order when viewed from the outside of the model** (i.e., from the direction the normal points toward).

## OBJ (Wavefront)

**Type:** mesh format

**What it is:** a more detailed mesh format which includes explicit vertex, normal, and texture-coordinate (system of numbers describing how a 2D image wraps onto a 3D model's surface, assigned to the model's vertices) lists with face definitions referencing them by index, i.e. vertices are declared once in a shared list, and faces reference these vertices by index (indexed geometry). Connectivity is more explicit, as shared edges point to the *same* vertex entry (as opposed to repeating coordinates, like in STL).

**Parsing:** same libraries for STL can be used, just need to pass in more parameters.

**Advantages over standard STL:** OBJ files can carry proper vertex normals and grouped sub-objects, so less reconstruction generally is required.

### OBJ File Format

Each line starts with a short tag describing what kind of data it contains.

```
# comment line
mtllib materials.mtl      # reference to a companion material file
o ObjectName              # named object/group
v 0.0 0.0 0.0             # vertex position
v 1.0 0.0 0.0
v 0.0 1.0 0.0
vt 0.0 0.0                # texture coordinate
vn 0.0 0.0 1.0            # vertex normal
usemtl MaterialName       # which material subsequent faces use
f 1//1 2//1 3//1          # face, referencing vertex/texture/normal indices
```

## Fields

- $v\ x\ y\ z\ [w]$ : vertex position. Three floats with an optional fourth (homogenous coordinate, rarely used, default=1.0).
- $vt\ u\ v\ [w]$ : a texture coordinate (UV mapping). Used for applying images/textures to the surface. Largely irrelevant (since for 3D printing, you don't care about visual texture as much) but needs to be handled by the parser.
- $vn\ n_x\ n_y\ n_z$ : a vertex normal declared explicitly and separately from vertex positions. This is the largest practical difference between OBJ and STL - normals are attached to *vertices*, not faces (STL triangles). Well-formed OBJ files can encode smooth, continuous normals across a curved surface, avoiding the faceted normals problem encountered by STL.
- $f\ \dots$  : a face definition.

## NURBS via CAD interchange

**Type:** parametric surface representation

**What it is:** true parametric surfaces, defined by *control points*, *knot vectors*, and *degree*.

They describe smooth, *curved* (not flat) patches exactly, no approximation. CAD interchange formats like STEP (.stp) and IGES (.igs) are how NURBS models get exported from tools like Rhino.

**Parsing:** OpenCASCADE (OCCT), standard open-source CAD geometry kernel with Python bindings through pythonocc-core.

## Exporting

The internally represented toolpaths must be exported to **high-level, proprietary programming languages** to control specific robots.

## ABB RAPID

Used to control ABB industrial robots.

Example RAPID motion instruction:

```
MoveL [[120.5,45.2,300.0],[0.707,0,0.707,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]],  
v100, z5, tExtruder;
```

Five Components:

### 1. Instruction Name

- a. MoveL - *linear interpolation* in Cartesian space (constant **tool center point (TCP)** speed along a straight line). Usually used for almost all non-planar deposition segments. Controlled, predictable tool motion along the surface.
- b. MoveJ - *joint interpolation*. Each axis moves to its target independently. TCP path is not straight nor predictable. Used for non-print travel motions (retract/reposition) where path shape doesn't matter.

- c. MoveC - *circular motion*. Requires two target points (via-point and end-point). Rarely needed unless segmentation strategy explicitly emits circular arcs.

## 2. RAPID pose representation: **robtarg**

- a. Robtarget record follows this format:  

$$[[x,y,z], [q1,q2,q3,q4], \text{confdata}, \text{extax}]$$
- b.  $[x,y,z]$  - *translation*. TCP position in mm, defined **relative** to the active work object's coordinate frame, not the world/base frame.
- c.  $[q1,q2,q3,q4]$  - *rotation*. Orientation as a quaternion in **w, x, y, z order**. Defines TCP orientation relative to the work object frame.
- d.  $\text{confdata}=[cf1,cf4,cf6,cfx]$  - *configuration data*. Given a target pose, there are (usually) multiple valid joint-angle solutions (elbow up/down, wrist flip, etc) that reach it. RAPID requires us to specify the solution.
  - i.  $cf1,cf4,cf6$  - *integer quadrant numbers*. Each represents which 90°-wide rotational quadrant that joint angle falls into ( $\text{quadrant}=\text{floor}(\text{joint\_angle}/90^\circ)$ ). The three axes (1, 4, 6) on a standard 6-axis arm can each rotate through more than one full quadrant to reach the same TCP pose - RAPID needs to know which one your solution used.
  - ii.  $cfx$  - *axis configuration number*. The number (0-7) encodes the elbow/shoulder/wrist branch of the **inverse kinematics (IK)** solution as a single index.

In practice, our K/IK module must compute actual joint angles for each waypoint, derive the integer quadrant numbers from them, and pick a consistent axis configuration number, which comes from wherever our reach/joint-limit solver already computes candidate configurations.

- e.  $\text{extax}=[\text{eax}_a, \dots, \text{eax}_f]$  - *external axes*. Values for up to 6 external axes (turntable, linear rail, etc), in the order the controller's system config assigns them. 9E9 is RAPID's sentinel for "axis not present/not used". If our cell has no external axes, all stay as 9E9 (as per the example), but if an axis is added to the kinematic model later on, its position goes here. This field accommodates for the "not precluding other robot platforms/axes later" architecture condition.

## 3. Speed data

- a.  $v_{100}$  - predefined speed record (100mm/s TCP speed with default orientation/external-axis speed caps).
- b. For finer control, speed records can be explicitly declared:  

$$[v_{\text{tcp}}, v_{\text{ori}}, v_{\text{leax}}, v_{\text{reax}}]$$
  - i.  $v_{\text{tcp}}$  - *TCP linear speed* in mm/s. Actual print speed, maps directly from toolpath's speed field.
  - ii.  $v_{\text{ori}}$  - *reorientation speed* in degrees/s. Caps how fast the tool can rotate while translating. On non-planar paths, since the tool's orientation continuously varies (unlike planar paths where the tool's orientation is fixed for each layer), it is worth exposing this field as a configurable parameter rather than hardcoding a default.
  - iii.  $v_{\text{leax}}, v_{\text{reax}}$  - *linear and rotational external-axis speeds*.

## 4. Zone data

# KUKA KRL

Used to control KUKA industrial robots.

# Visualisation

All these are open source python libraries

Viser – Web based viewer. Visualise point clouds, meshes with programmatic camera control and rendering. Uses transform gizmos, object selection and click events to maneuver around the 3D environment. Hosts a local server with a client interface and can be run over SSH. Considered fast and simple

- API for visualizing 3D primitives
- GUI building blocks: buttons, checkboxes, text inputs, sliders, etc.
- Scene interaction tools (clicks, selection, transform gizmos)
- Programmatic camera control and rendering
- An entirely web-based client, for easy use over SSH!
- Jupyter notebook compatibility

Rerun – Runs on web and desktop. Uses timeline screening playback to analyse states frame by frame and can view key math components such as orientation vectors, velocities etc. Great for toolpath simulation

PyVista – Desktop (VTK-based). Can display 3d meshes, stress points and contour maps. Can integrate with numpy elements and is efficient with numerical computations.

MesbLib – Supports multi axis workflows and manufacturing simulations. Backed by powerful geometry engines to perform additive manufacturing, collision engine logic and mesh slicing